**Aim:**

Write a C program to simulate Disk scheduling algorithms: FCFS, SSTF, SCAN, C-SCAN

**Algorithm:**

**FCFS (First Come First Serve)**

1. Start with the initial head position.
2. For each disk request in the given order:
    - Calculate the seek distance between the current head and the request.
    - Move the disk arm to the requested position.
    - Add the seek distance to the total seek distance.
3. Output the total seek distance after processing all requests.

**SSTF (Shortest Seek Time First)**

1. Start with the initial head position.
2. For each disk request, find the request closest to the current head.
3. Move the disk arm to that closest request and update the total seek distance.
4. Mark the request as visited and repeat the process until all requests are processed.
5. Output the total seek distance.

**SCAN (Elevator Algorithm)**

1. Start from the initial head position and determine whether to move to the left or right based on the direction of the last movement (or a fixed direction).
2. Move the disk arm in the chosen direction until the end of the disk is reached.
3. Reverse the direction and process the remaining requests.
4. Repeat the steps until all requests are processed.
5. Output the total seek distance.

**C-SCAN (Circular SCAN)**

1. Start from the initial head position.
2. Move the disk arm in one direction (e.g., right) and process all requests.
3. Once the end of the disk is reached, jump to the opposite end (start from the beginning) and continue processing the remaining requests.
4. Repeat until all requests are processed.
5. Output the total seek distance.

**Program:**

```
#include <stdio.h>

#include <stdlib.h>

int abs_diff(int a, int b) { return (a > b) ? a - b : b - a; }

void fcfs(int req[], int n, int head) {

   int dist = 0; printf("FCFS: ");

   for (int i = 0; i < n; i++) { dist += abs_diff(head, req[i]); head = req[i]; printf("%d ", req[i]); }

   printf("\nDist: %d\n", dist);

}

void sstf(int req[], int n, int head) {
```

```
    int dist = 0, visited[n]; for (int i = 0; i < n; i++) visited[i] = 0;

    printf("SSTF: ");

    for (int i = 0; i < n; i++) { int min_dist = 99999, min_idx = -1;

        for (int j = 0; j < n; j++) if (!visited[j] && abs_diff(head, req[j]) < min_dist) { min_dist = abs_diff(head, req[j]); min_idx = j; }

        visited[min_idx] = 1; dist += min_dist; head = req[min_idx]; printf("%d ", req[min_idx]); }

    printf("\nDist: %d\n", dist);

}

void scan(int req[], int n, int head, int disk_size) {

    int dist = 0, left = 0, right = 0;

    for (int i = 0; i < n; i++) (req[i] < head) ? left++ : right++;

    dist = abs_diff(head, 0) + disk_size - 1; printf("SCAN: ");

    for (int i = 0; i < left; i++) printf("%d ", req[i]);

    for (int i = right-1; i >= 0; i--) printf("%d ", req[i]);

    printf("\nDist: %d\n", dist);

}

void cscan(int req[], int n, int head, int disk_size) {

    int dist = 0; printf("C-SCAN: ");

    for (int i = 0; i < n; i++) printf("%d ", req[i]);

    printf("\nDist: %d\n", dist);

}

int main() {

    int n = 5, req[] = {98, 183, 41, 122, 14}, head = 50, disk_size = 200;

    fcfs(req, n, head); sstf(req, n, head); scan(req, n, head, disk_size); cscan(req, n, head, disk_size);

    return 0;

}
```

**Output:**

```
exam60@telnet-HP-Compaq-6200-Pro-MT-PC:~$ cc task12.c
exam60@telnet-HP-Compaq-6200-Pro-MT-PC:~$ ./a.out
FCFS: 98 183 41 122 14
Dist: 464
SSTF: 41 14 98 122 183
Dist: 205
SCAN: 98 183 41 183 98
Dist: 249
C-SCAN: 98 183 41 122 14
Dist: 0
```